Recurrent Neural Network

Dr. Satish R. Sankaye

Outline

- Sequential Learning & it's Problem
- Introduction to Recurrent Neural Network
- Architecture of RNN
- Types of RNN
- Application of RNN
- Advantage of RNN
- Limitation of RNN

Sequential Learning & it's Problem

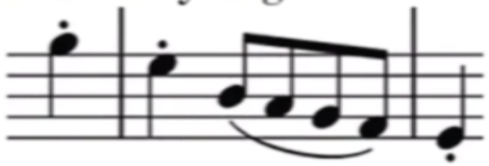
- **Sequence data** refers to a type of data where the order of elements is significant, and the elements are typically arranged in a specific sequence or chronological order. In sequence data, each element in the sequence is related to the ones that come before and after it, and the way these elements are ordered provides context, meaning, and structure to the data.
- It includes textual data, time series data, audio signals, video streams or any other ordered data.

Sequential Learning & it's Problem

- **Order Matters:** In sequence data, the arrangement of elements is important. Changing the order of elements can lead to different meanings or interpretations. For example, in natural language processing, the order of words in a sentence conveys the sentence's meaning.
  - CAT CHASE RAT, RAT CHASE CAT
- **Temporal or Sequential Relationship:** Sequence data often exhibits a temporal or sequential relationship. This means that the elements are typically collected or observed in a specific order over time. Time series data such as stock price, for instance, records values at different time points.
- **Variable Length:** Sequences can have variable lengths. In natural language, sentences can be of different lengths, and in genomics, DNA sequences can vary in length depending on the organism.

Application of RNN

Examples of sequence data

	x		y
Speech recognition		→	"The quick brown fox jumped over the lazy dog."
Music generation	∅	→	
Sentiment classification	"There is nothing to like in this movie."	→	★☆☆☆☆
DNA sequence analysis	AGCCCCTGTGAGGAACTAG	→	AGCCCCTGTGAGGAACTAG
Machine translation	Voulez-vous chanter avec moi?	→	Do you want to sing with me?
Video activity recognition		→	Running
Name entity recognition	Yesterday, Harry Potter met Hermione Granger.	→	Yesterday, Harry Potter met Hermione Granger.

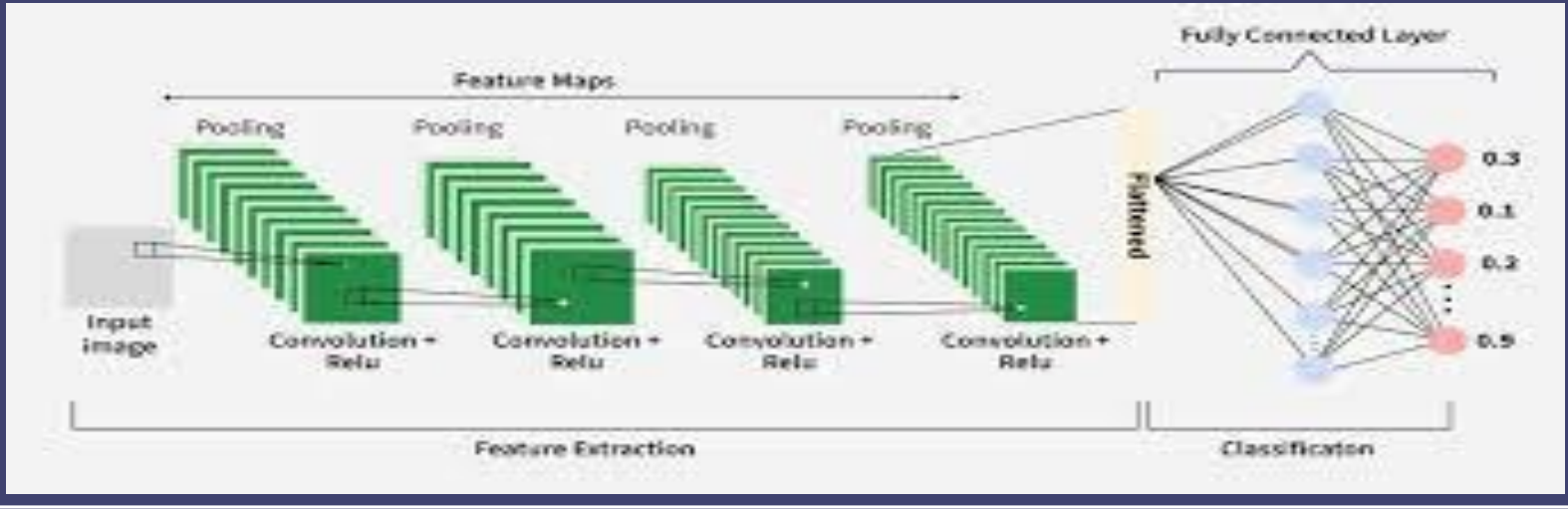
Introduction to RNN

- Feedforward NN Process Data:
  - Process each row independently
  - Does not remembers what it saw previously
  - Does not care about the order of the samples. It does not matter if row 1 comes before row 2 or after.
  - Expects fixed length input vectors.

_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa
...	...	...	...	...
6.7	3.0	5.2	2.3	virginica
6.3	2.5	5.0	1.9	virginica
6.5	3.0	5.2	2.0	virginica
6.2	3.4	5.4	2.3	virginica
5.9	3.0	5.1	1.8	virginica

Introduction to RNN

- CNN Process Data:
  - Size of Input is always fixed. (eg. Image size can be 64x64 or 128x128)
  - CNN too treats each input independently of all others.
  - Also prediction depends on the current image and never on previous inputs.



Introduction to RNN

We need to build the model for such applications where :

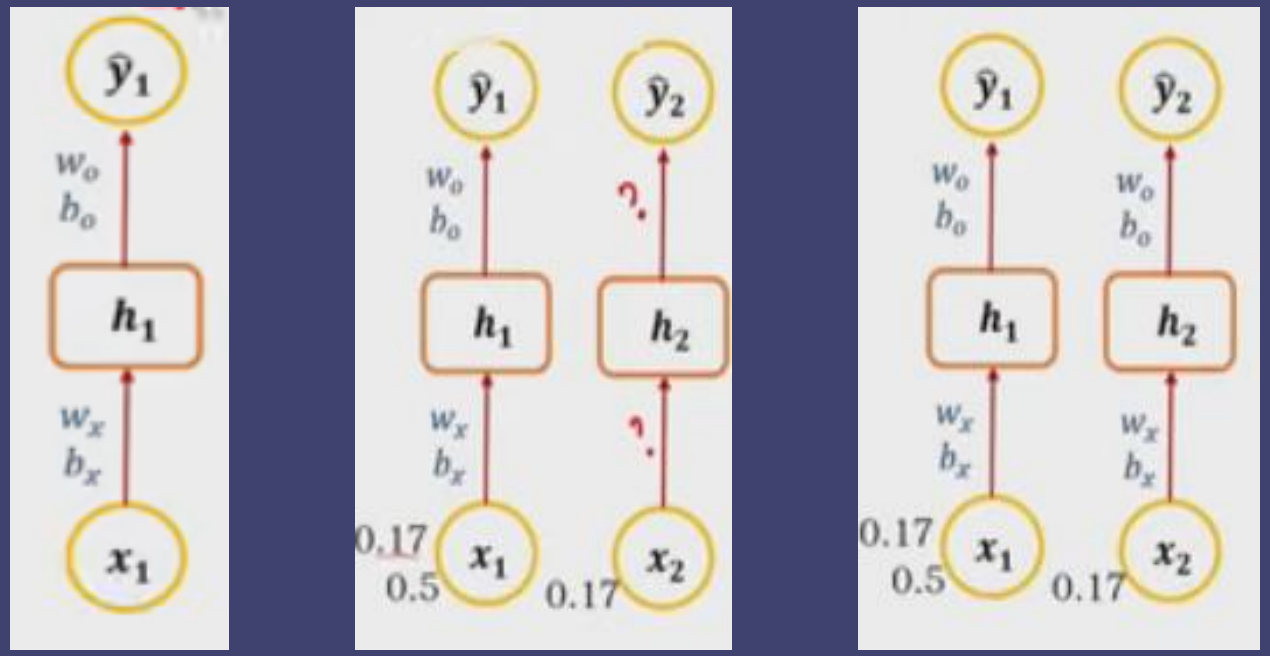
1. The model must **handle dependencies between the inputs.**
2. The model must **handle variable-length inputs.**

- To achieve (1) and (2), at each time step, the **function** executed by the model should be **same**.
- We Perform the first steps as:

$$X_{norm} = X - \min(x) \ / \max(x) - \min(x)$$

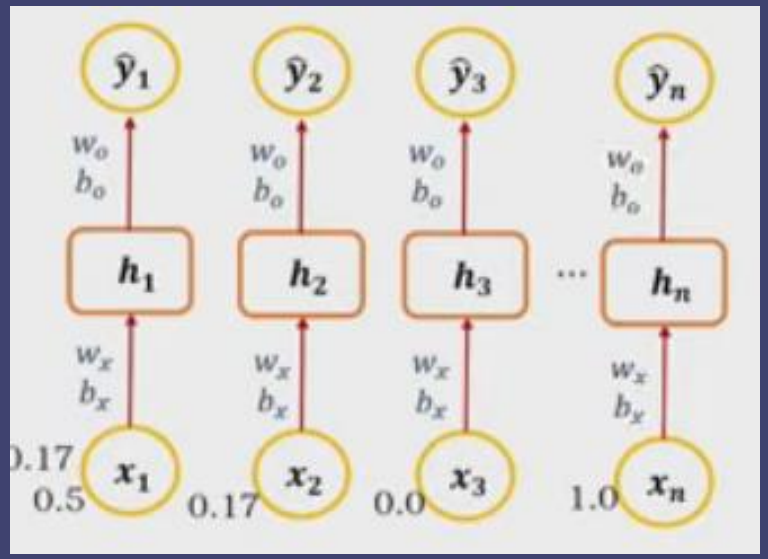
Day	Val	X	Y
Mon	9	0.5	0.17
Tue	7	0.17	0.0
Wed	6	0.0	0.67
Thurs	10	0.67	0.33
Fri	8	0.33	0.17
Sat	7	0.17	0.83
Mon	11	0.83	0.5
Tue	9	0.5	0.0
Wed	6	0.0	1.0
Thurs	12	1.0	

Introduction to RNN



Day	Val	X	Y
Mon	9	0.5	0.17
Tue	7	0.17	0.0
Wed	6	0.0	0.67
Thurs	10	0.67	0.33
Fri	8	0.33	0.17
Sat	7	0.17	0.83
Mon	11	0.83	0.5
Tue	9	0.5	0.0
Wed	6	0.0	1.0
Thurs	12	1.0	

Introduction to RNN



$$h_1 = tanh(x_1 w_x + b_x)$$

$$\hat{y}_1 = f(h_1 w_o + b_o)$$

$$h_1 = tanh(x_1 w_x + b_x) \qquad \hat{y}_1 = f(h_1 w_o + b_o)$$

$$h_2 = tanh(x_2 w_x + b_x) \qquad \hat{y}_2 = f(h_2 w_o + b_o)$$

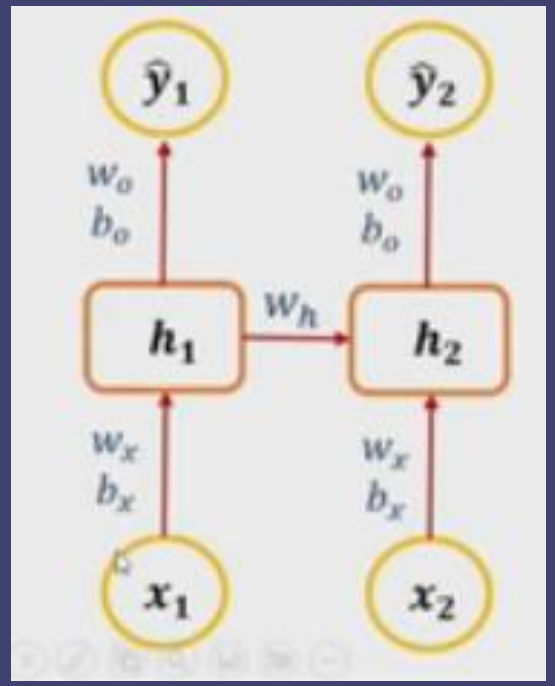
$$h_i = tanh(x_i w_x + b_x)$$

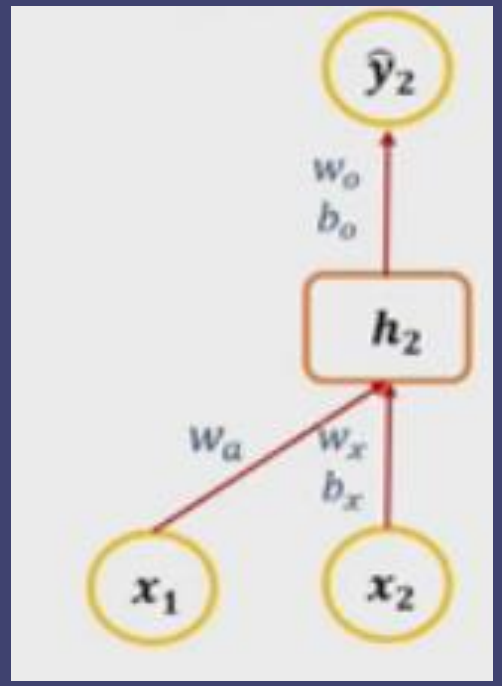
$$\hat{y}_i = f(h_i w_o + b_o)$$

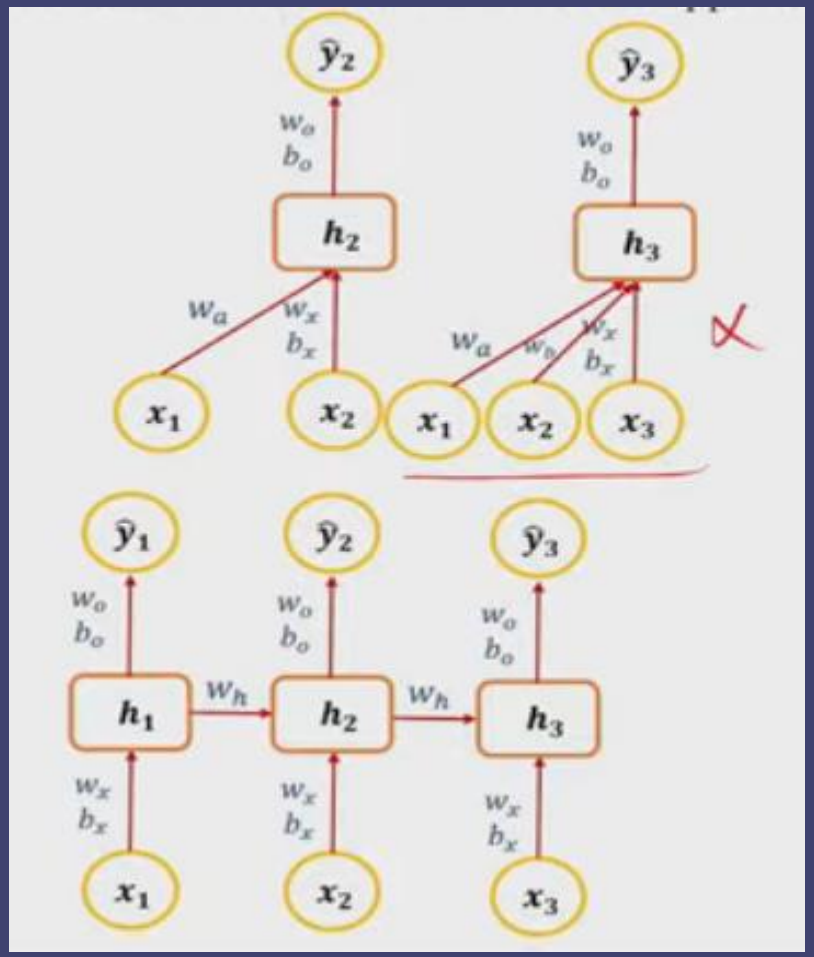
Day	Val	X	Y
Mon	9	0.5	0.17
Tue	7	0.17	0.0
Wed	6	0.0	0.67
Thurs	10	0.67	0.33
Fri	8	0.33	0.17
Sat	7	0.17	0.83
Mon	11	0.83	0.5
Tue	9	0.5	0.0
Wed	6	0.0	1.0
Thurs	12	1.0	

Architecture of RNN

- Handle dependencies between inputs.
- Function should be the same.

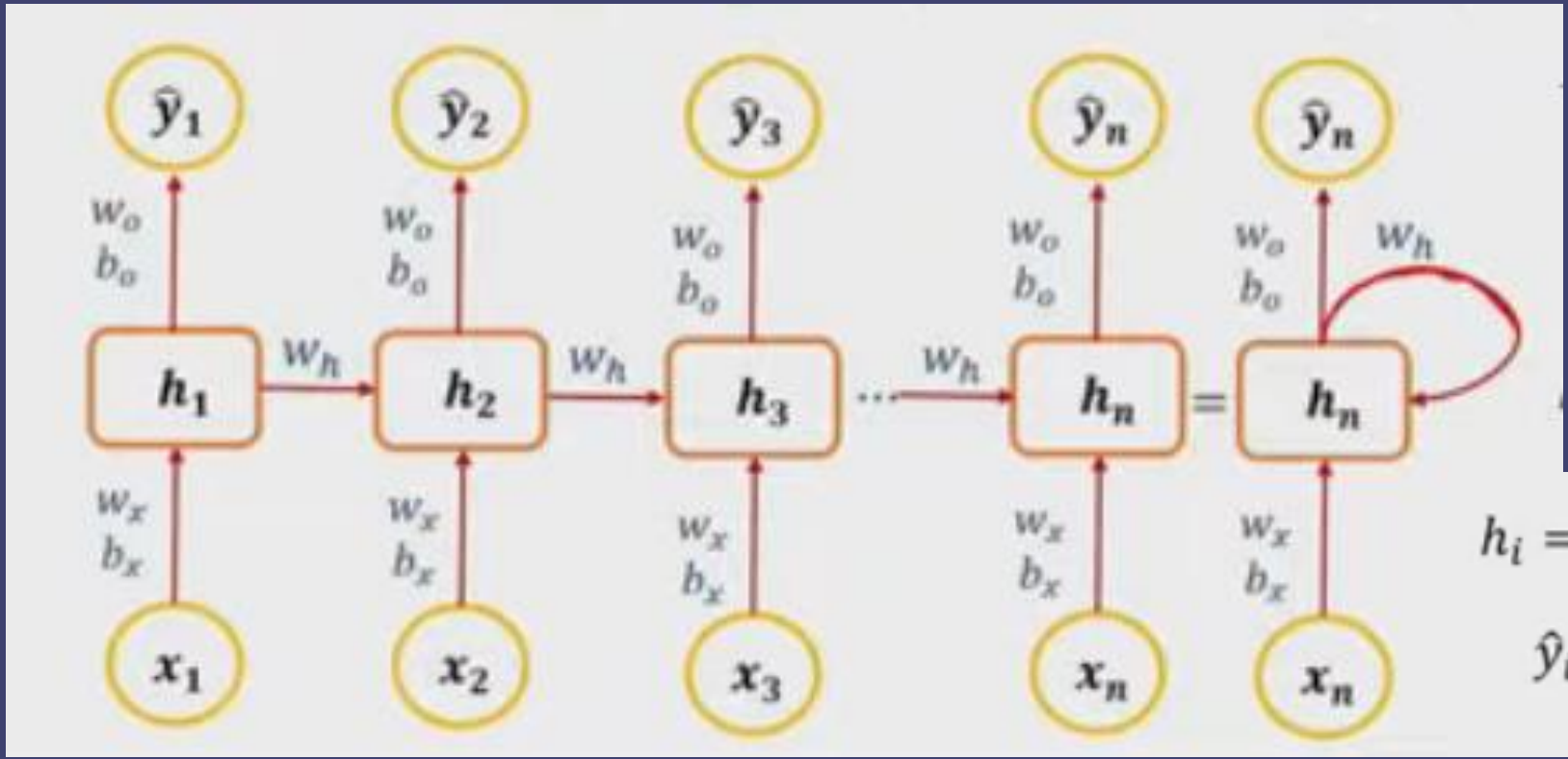






Architecture of RNN

- **Recurrent Connection in the Network, thus RNN**



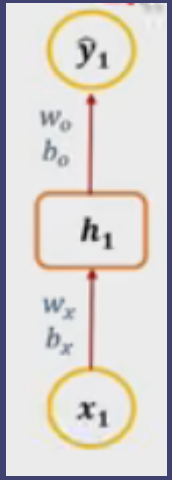
$$h_i = tanh(x_i w_x + h_{i-1} w_h + b_x)$$

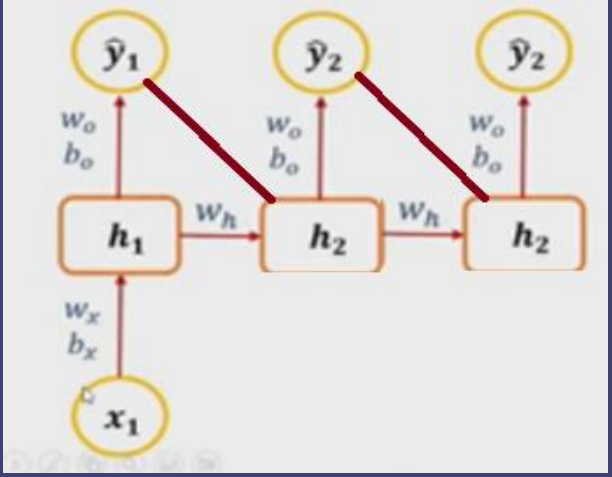
$$\hat{y}_i = f(h_i w_o + b_o)$$

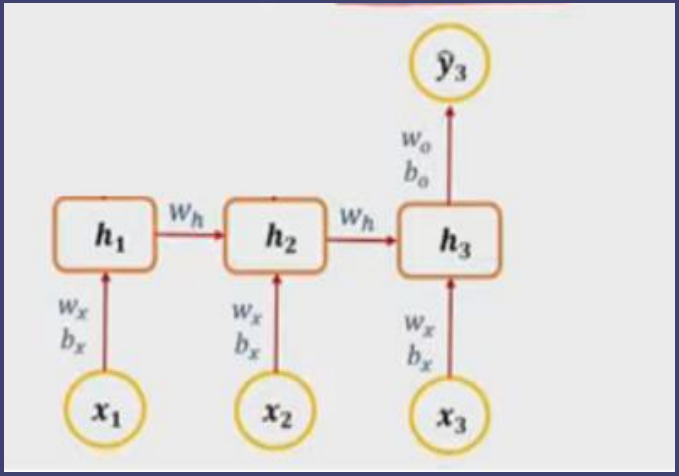
Types of RNN

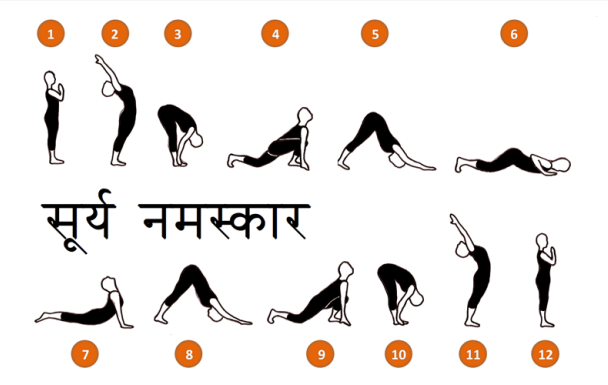
- RNN is of four types:
  - One to One
  - One to Many
  - Many to One
  - Many to Many

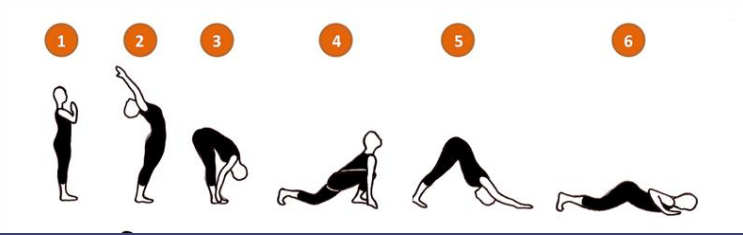
Types of RNN


$\hat{y}_1$
w_o
b_o
h_1
w_x
b_x
x_1


$\hat{y}_1$
$\hat{y}_2$
$\hat{y}_2$
w_o
b_o
h_1
w_h
h_2
w_h
h_2
w_x
b_x
x_1


$\hat{y}_3$
w_o
b_o
h_1
w_h
h_2
w_h
h_3
w_x
b_x
x_1
x_2
x_3


1
2
3
4
5
6
सूर्य नमस्कार
7
8
9
10
11
12


1
2
3
4
5
6

Advantageous of RNN

- **Sequential Data Processing:** Highly effective for tasks involving time series, natural language processing (NLP), and speech recognition.
- **Memory of Past Inputs:** RNNs use hidden states to retain contextual information from previous steps in a sequence. This enables RNNs to capture and utilize context from previous time steps, to model dependencies and relationships within sequences.
- **Variable-Length Inputs:** Can process sequences of variable length as opposed to traditional neural networks that require fixed-sized input vectors. This makes them particularly useful in applications where input data is not fixed in size.
- **Parameter Sharing:** The same weights are used across all time steps, reducing model size compared to storing parameters for each step. This is efficient in terms of memory and computational requirements.

Limitation of RNN

- **Vanishing/Exploding Gradient Problems:** During training, gradients can become too small or too large, making it difficult for the model to learn long-term dependencies.
- **Difficulty with Long-Term Dependencies:** Due to the vanishing gradient, vanilla RNNs struggle to retain information from early steps, limiting their effectiveness on long sequences.
- **Slow Training Time:** Because they are sequential, they cannot process data in parallel, leading to high computational costs and slow training.
- **No Future Context:** Standard RNNs only consider past and current inputs, ignoring future context which is sometimes necessary for understanding.
- **Difficulty with Parallelization:** RNNs are not able to take complete advantages of modern hardware architectures such as GPUs designed for parallel processing.